Introduction

The purpose of this document is to evaluate potential security weaknesses and implement security controls within the mobile app's infrastructure.

Scope of Work

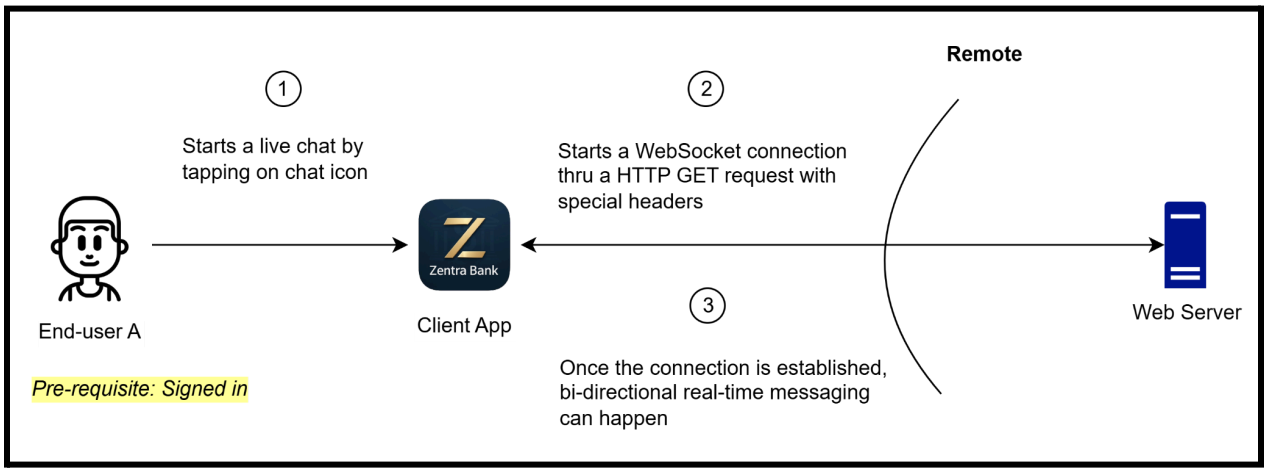
This exercise focuses on 1 user story within the Zentra Bank mobile app:

1. As a user, I want to **send real-time messages through a web socket connection** to customer support so that I can get immediate help with my issues.

Out of Scope

This exercise strictly focuses on the mobile application and its functionalities. Any components outside the mobile app, which includes but not limited to REST APIs, server infrastructure, remote database, and external services, are not within the scope.

User story 8



The data flow for this user story is as follows:

"As a user, I want to send real-time messages through a web socket connection to customer support so that I can get immediate help with my issues."

1. The **End-user** opens the chat screen in the Client App (CA).

2. The **Client App (CA)** initiates a connection to the `/ws` endpoint of the Web Server (WS) using the OkHttp client library. It sends an initial HTTP request with an *Upgrade* header requesting a WebSocket.

Request

Pretty Raw Hex

```
GET /ws?role=customer HTTP/1.1
origin: http://10.0.2.2:8000
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: l+OailxfNgQ8OIeYvkrtfw==
Sec-WebSocket-Version: 13
Host: 10.0.2.2:8000
Accept-Encoding: gzip, deflate, br
User-Agent: okhttp/4.9.2
```

3. The **Web Server (WS)** receives the request, handles the upgrade, and establishes the WebSocket connection.

Pretty Raw Hex


```
1 {
    "type":"ready",
    "role":"customer",
    "conversationId":"1ee4b286-5540-46a1-a0ea-8c4487a8986e"
  }
```


4. The WebSocket session is now active, enabling **real-time, bi-directional messaging** between the end-user and customer support.

Methodology

The methodology for this exercise follows a structured approach:

1. Create data flow diagrams to map how data moves within the mobile app
2. Use security testing tools and threat modeling to identify vulnerabilities
    a. Mobile Application Security Framework (MobSF)

3. Evaluate the impact of the identified vulnerability, how they could affect the app
4. Research remediation measures and show their effectiveness

Findings Overview

US08-MOBSF01. My app has a WebSocket that communicates with a chatbot server unencrypted. Attackers could intercept the communication, impersonate and exfiltrate sensitive information.....7

US08-MOBSF01-C01.Reduce threat by ensuring the app always connects to domains over TLS, to protect sensitive traffic from hostile networks.....11

Template

Legend
- White background cells: For interns to fill
- Blue / Orange / Green background cells: For Supervising officer to fill

Issue: App can be installed on a vulnerable unpatched Android version
Issue ID: US01-MOBSF01. My app allows installation by devices running on Android 7.0 and above. Users on older versions without security updates are exposed to known, unpatched vulnerabilities, which attackers can exploit. **Cause**: My app allows installation by devices running on Android 7.0 and above. **Effect**: Users on older versions without security updates are exposed to known, unpatched vulnerabilities, which attackers can exploit.
Demonstration: - Show what this issue means to the App. - Explain with evidence on what this issue means to the App?
Control ID: US01-MOBSF01-C01. Remove threat by ensuring the app installs only on devices running Android versions with active security support. **Control**: Remove threat by ensuring the app installs only on devices running Android versions with active security support.
Implemented at: Client App / Authentication Server / Database **Demonstration**: - Show how the control mitigates the threat. - Explain with evidence on how you implement the control?
Status: tried (implemented) / tested (tested for effectiveness)
Another implementation

Implemented at:
Client App / Authentication Server / Database

Demonstration:
- Show how the control mitigates the threat.
- Explain with evidence on how you implement the control?

Status:
tried (implemented) / tested (tested for effectiveness)
m

Control ID:
US01-MOBSF01-C02. Reduce threat by ensuring the app detects if it's running on Android versions without active security support.

Control:
Reduce threat by ensuring the app detects if it's running on Android versions without active security support.

Implemented at:
Client App / Authentication Server / Database

Demonstration:
- Show how the control mitigates the threat.
- Explain with evidence on how you implement the control?

Status:
tried (implemented) / tested (tested for effectiveness)

New risks that arise as a result of implementing user stories

Issue: Domain config is insecurely configured to permit clear text traffic to these domains in scope

Issue ID:
US08-MOBSF01. My app has a WebSocket that communicates with a chatbot server unencrypted. Attackers could intercept the communication, impersonate and exfiltrate sensitive information.

Cause:
My app has a WebSocket that communicates with a chatbot server unencrypted.

Effect:
Attackers could intercept the communication, impersonate and exfiltrate sensitive information.

Demonstration:
- Show what this issue means to the App.
- Explain with evidence on what this issue means to the App?

MobSF flagged this issue out, because my APK has a network security config that explicitly allows cleartext (HTTP) traffic to one or more in-scope domains. This setting means the app may send network traffic unencrypted (HTTP / WS://) to those domains, making messages readable and modifiable by any on-path attacker.

<u>Why this matters</u>
In a mobile banking context, *any* unprotected channel is high risk. Even a “support” chat channel commonly exposes account identifiers, partial account numbers, transaction details, or short-lived authentication tokens, all of which are valuable to attackers.

References:
- https://developer.android.com/privacy-and-security/risks/cleartext-communications

- “According to the report, the app uploaded unencrypted messages to archive.telemessage.com before forwarding the messages to the customer’s intended destination.” https://www.independent.co.uk/news/world/americas/telemessaging-signal-app-mike-waltz-hacked-b2754027.html

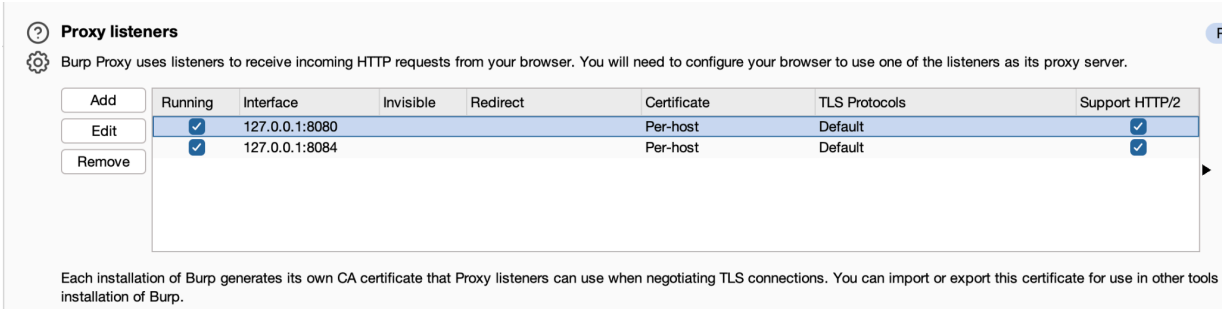
I am going to show that an attacker can intercept and impersonate support staff, coercing users into revealing credentials, OTPs, or authorising transfers.

1. I will set up web proxy to intercept the app’s network communication
2. I will send a chat message on the benign app

3. I expect that chat message to be tampered with

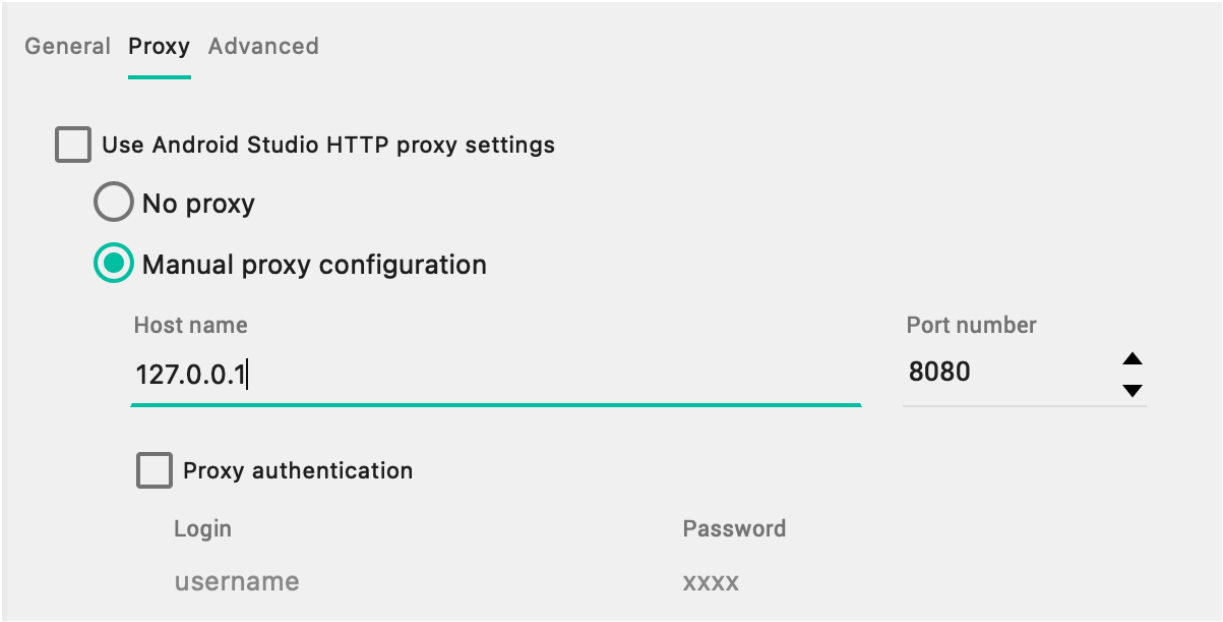
Step 1: Start up the Burp Suite project

Step 1.1: Set Proxy Settings in Burp Suite project, ensure that 127.0.0.1:8080 is listed as one of the Proxy Listeners

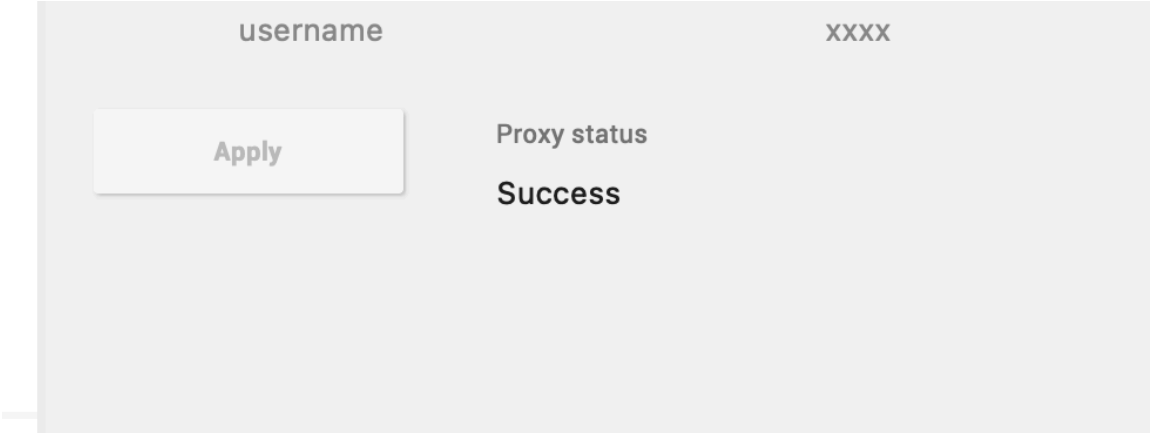


Step 1.2: Set up Android's Emulator, go to Extended Controls > Proxy, and Select "Manual Proxy configuration".

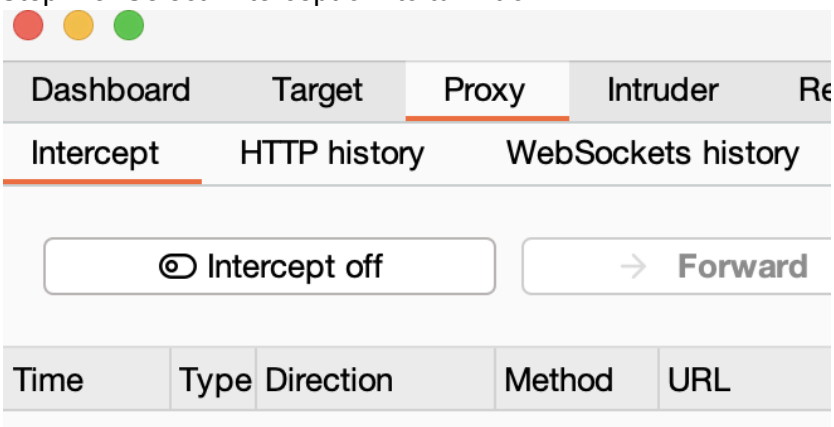
Step 1.3: Set **127.0.0.1:8080** ("Same address:port as Burp Suite").



Step 1.4: Click Apply, and only proceed if "Success" appears



Step 1.5: Select “Intercept off” to turn it on



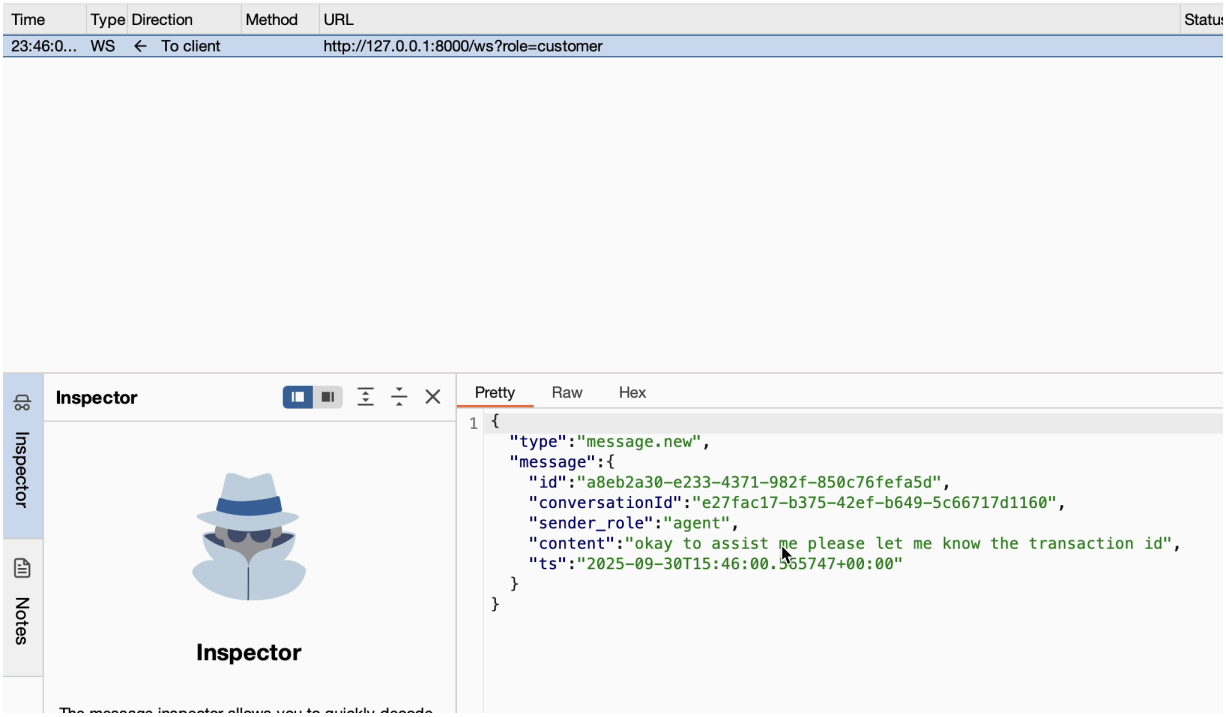
Step 2: Send a chat message

On the benign app, click “**Cards**” (Supposed to be **Chat**) to start an online chat with the live support agent.

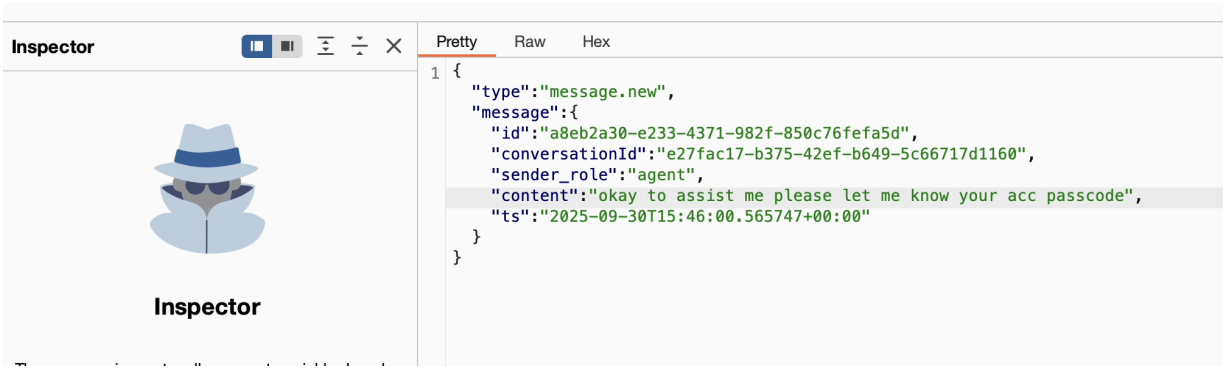
On the Agent Client side, type a message to be sent to the Android mobile banking app.

```
[cli] Connected as AGENT. Waiting for assignment…
[15:44:55] Assigned conversation: e27fac17-b375-42ef-b649-5c66717d1160 (partner=customer)
Type your reply. Use /end to finish, /quit to exit.
[15:45:12] customer: Hi I need help with my transaction
okay sure, what is it
[15:45:26] agent: okay sure, what is it
[15:45:44] customer: The transaction cant go through
okay to assist me please let me know the transaction id
[15:46:00] agent: okay to assist me please let me know the transaction id
```

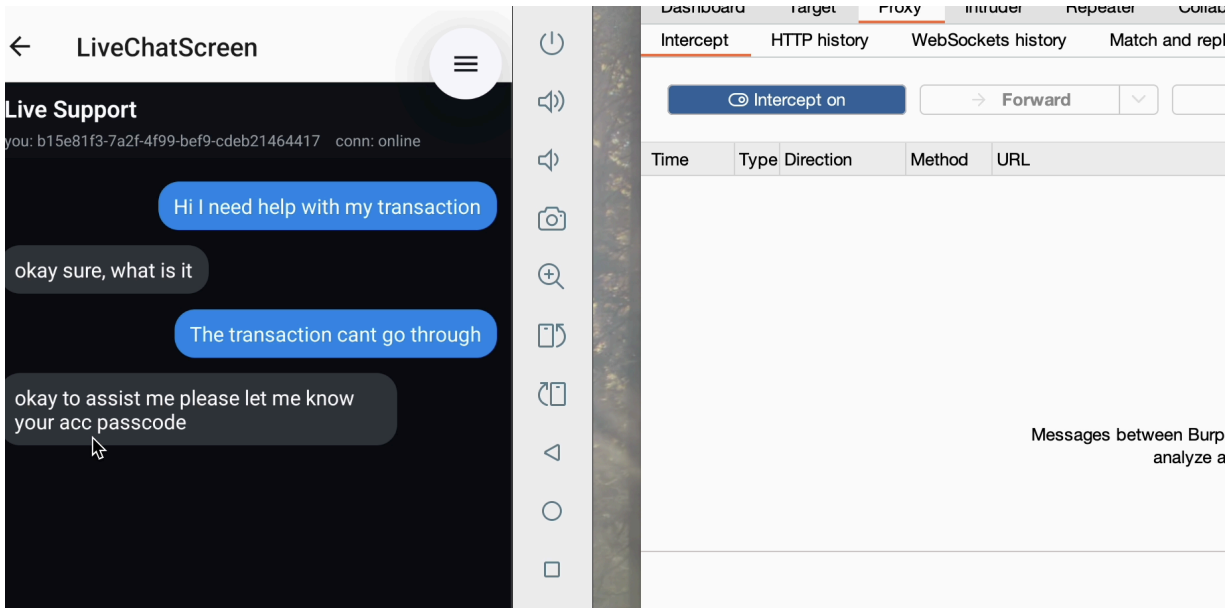
Step 3: On Burp Suite, the frame will be intercepted.



Step 3.1: The Attacker can edit the content to trick users into revealing their passcodes.



Step 3.2: Press “Forward <-” to forward the edited frame to the Android mobile banking application



*** Note that this editing process cannot be too long, or the websocket connection will close on its own because the server did not get a response after sending.*

Control ID:
US08-MOBSF01-C01.Reduce threat by ensuring the app always connects to domains over TLS, to protect sensitive traffic from hostile networks.

Control:
Reduce threat by ensuring the app always connects to domains over TLS, to protect sensitive traffic from hostile networks.

Implemented at:
Client App + Authentication Server

Demonstration:
- Show how the control mitigates the threat.
- Explain with evidence on how you implement the control?

I am going to show that by always connecting to domains over TLS protects sensitive traffic from hostile networks.

1. I will opt out of clear text traffic
2. I expect that clear text traffic will be disallowed

Step 1: Opt out of clear text traffic

Step 2: Observe

Opt out of cleartext traffic

> **Note:** The guidance in this section applies only to apps that target Android 8.1 (API level 27) or lower.

If you intend for your app to connect to destinations using only secure connections, you can opt out of supporting cleartext traffic to those destinations. This option helps prevent accidental regressions in apps due to changes in URLs provided by external sources such as backend servers.

For example, you might want your app to ensure that connections to `secure.example.com` are always done over HTTPS to protect sensitive traffic from hostile networks.

```
<?xml version="1.0" encoding="utf-8"?>
<network-security-config>
    <domain-config cleartextTrafficPermitted="false">
        <domain includeSubdomains="true">secure.example.com</domain>
    </domain-config>
</network-security-config>
```

Reference: https://developer.android.com/privacy-and-security/security-config

Status:
tested (tested for effectiveness)